Introduction to Neural Networks

Machine Learning with Python — Lecture 2, Session 1 (Oct 20)

EUGLOH — *Problem Solving Using Open-Source Languages; R and Python*

University of Novi Sad

Today's session

- **Logistic regression** — the sigmoid and the linear decision boundary
- **The artificial neuron** — a weighted sum plus a non-linear activation
- **Activation functions** — the non-linearity that makes networks powerful
- **The XOR problem** — the limitation of a single linear unit
- A brief **history** of neural networks — and the two AI winters

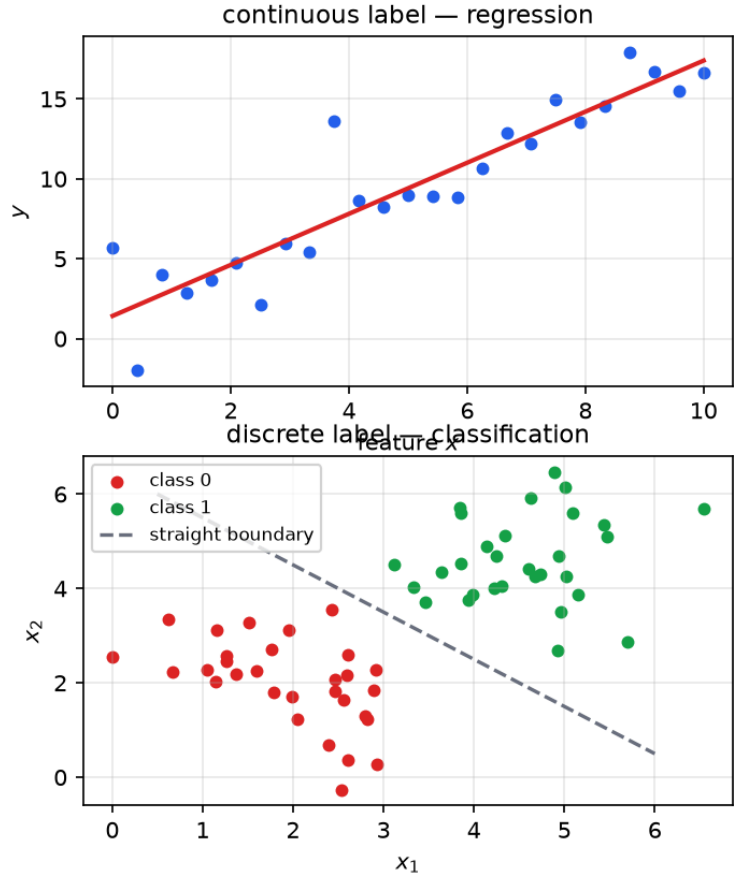
Session 1 of 4 today.

Part 1 — The basics

Introduction

- We can regress with a **straight line** (top right: a *continuous* label) and classify with **axis-aligned cuts** (trees).
- Now we classify with a straight line too — **logistic regression** (bottom right: a *discrete* label) — and see where that runs out.
- Many real problems need **smooth, curved boundaries**.

So we build up to **non-linear classifiers**,



starting from the logistic unit and ending at the neural network.

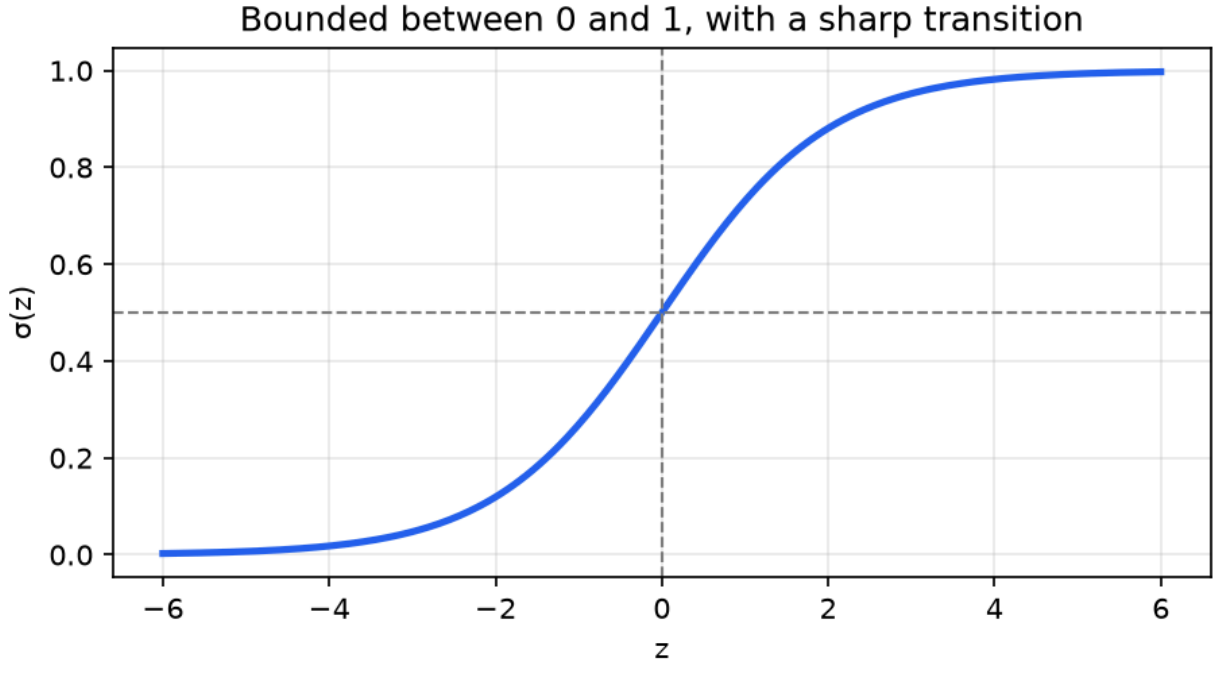
The logistic function

For classification the labels take only **two values** (0 and 1), so we want a function bounded between 0 and 1 with a sharp transition — the **logistic (sigmoid) function**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

In plain words: feed in **any** number z and σ bends it into something between 0 and 1. A large positive z gives almost 1; a large negative z gives almost 0; and $z = 0$ gives exactly 0.5.

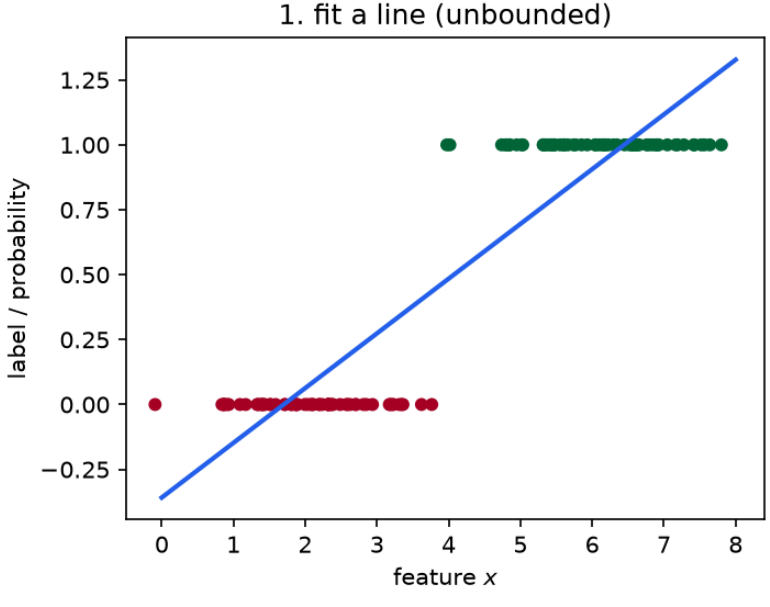
It is smooth everywhere, which is what lets us train it with gradient descent.

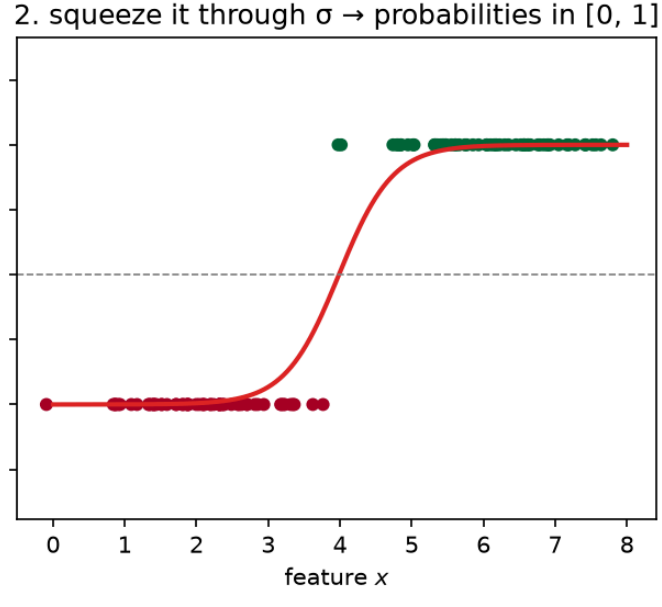

Bounded between 0 and 1, with a sharp transition
1.0
0.8
0.6
0.4
0.2
0.0
σ(z)
−6
−4
−2
0
2
4
6
z

A first intuition: compress the line into [0, 1]

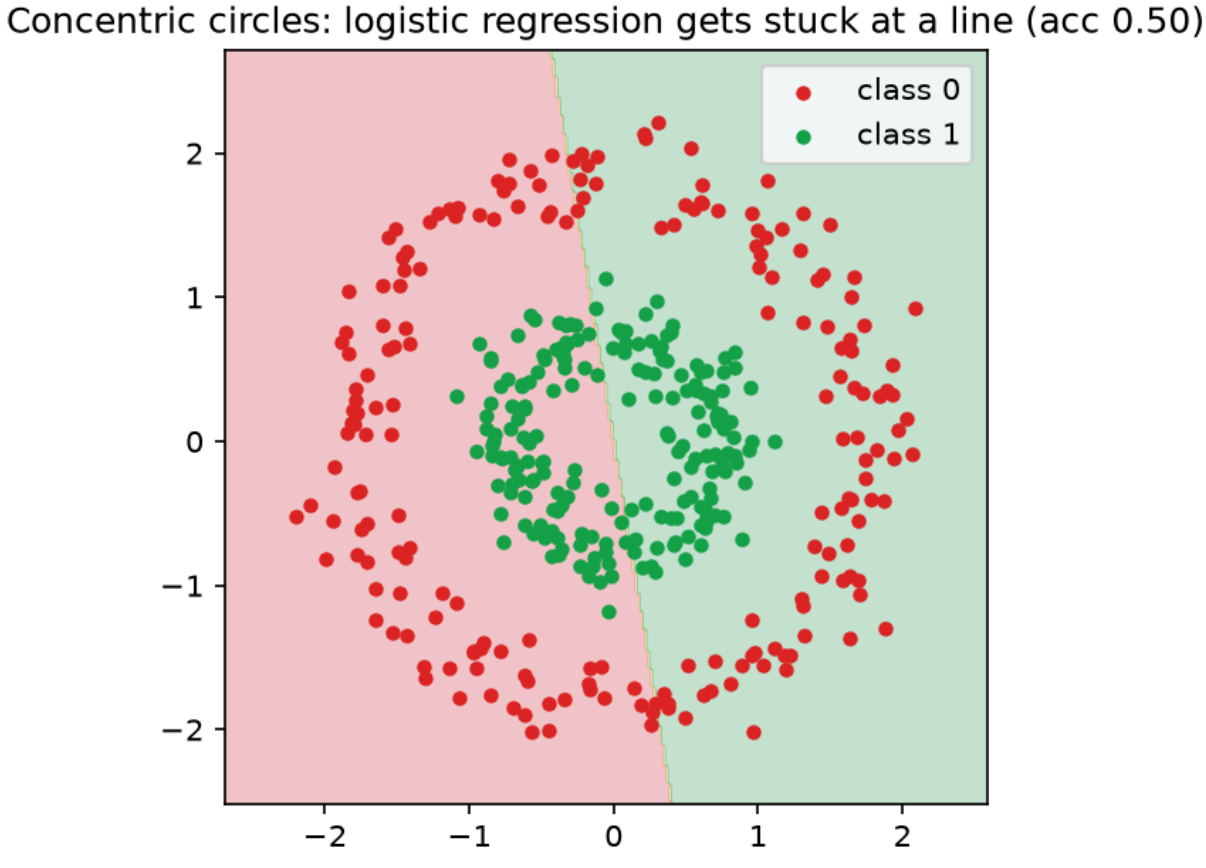
Take the linear model $z = w^\top x + b$, then **squeeze** its output through the sigmoid: $x \to w^\top x + b \to \sigma(w^\top x + b)$.

- far left: $\sigma \approx 0$ — confidently class 0
- far right: $\sigma \approx 1$ — confidently class 1
- in between: a probability!


1. fit a line (unbounded)
label / probability
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
0
1
2
3
4
5
6
7
8
feature x


2. squeeze it through σ → probabilities in [0, 1]
0
1
2
3
4
5
6
7
8
feature x

Because the boundary is a line, logistic regression fails whenever the classes are **not linearly separable** — here two concentric circles, where no straight line can separate them.



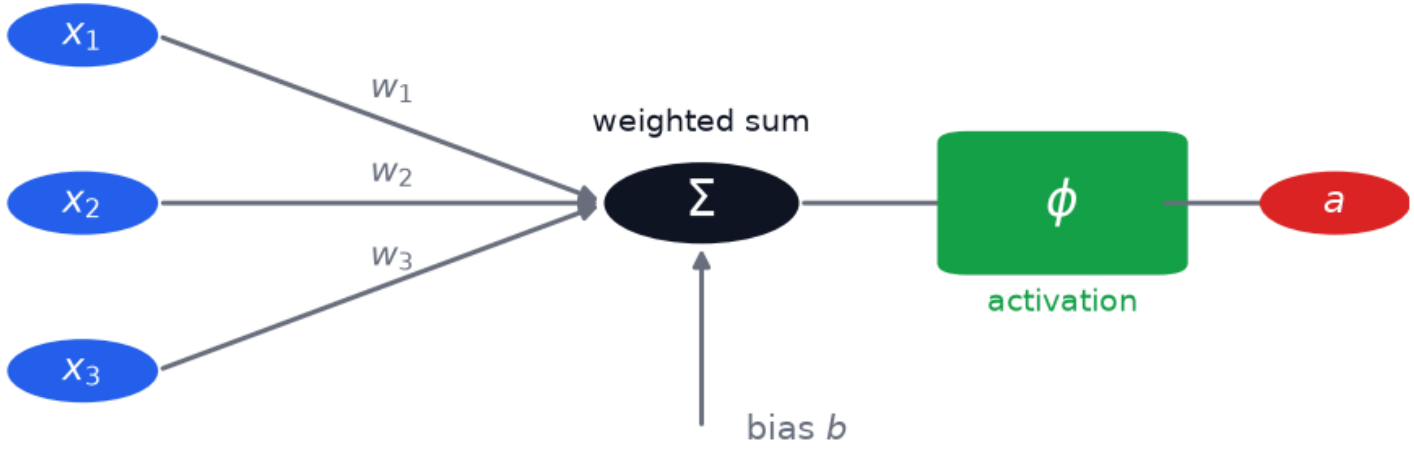
The artificial neuron

A neuron first forms a **weighted sum** of its inputs, then squeezes the result through a non-linear **activation**:

$$z = w_1 x_1 + w_2 x_2 + \cdots + b, \qquad a = \phi(z)$$

In plain words: multiply each input by its **weight**, add them up (plus a **bias** b), then pass the total through a curve ϕ.

The **perceptron** is exactly this unit with a threshold/sigmoid activation. Recognise it? **Logistic regression is exactly one neuron** with the sigmoid activation — the model we opened this session with. A neural network is *many neurons stacked in layers*, each feeding the next.

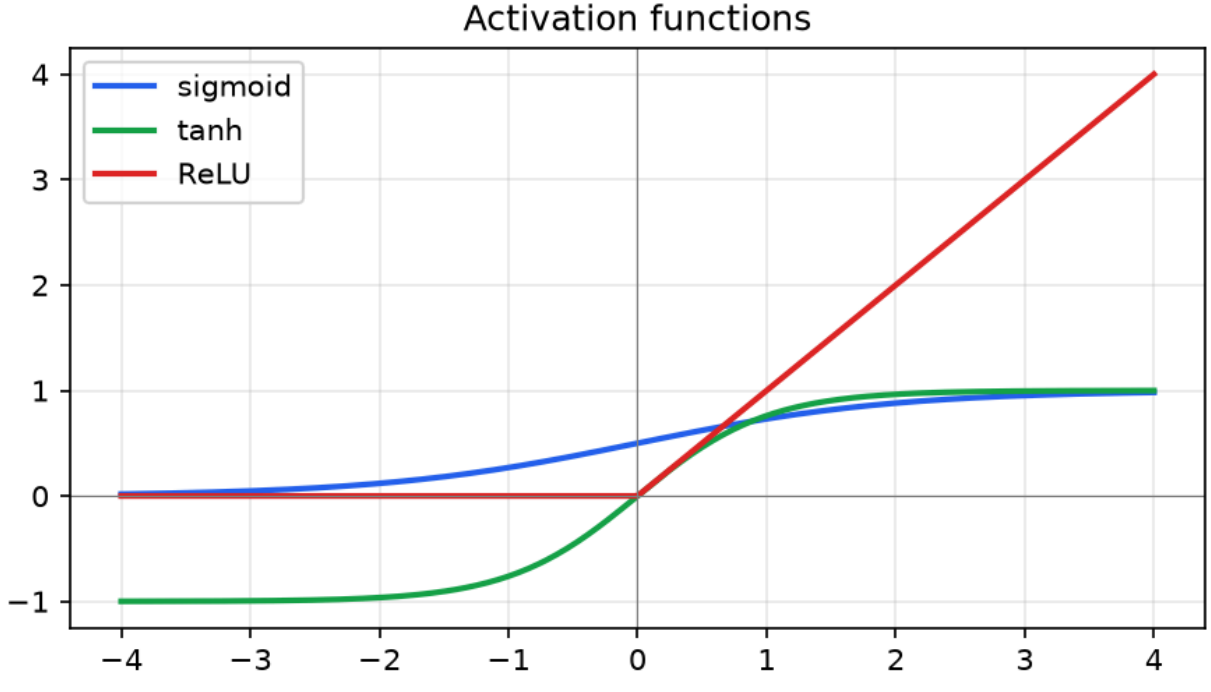

x_1
x_2
x_3
w_1
w_2
w_3
weighted sum
Σ
ϕ
activation
a
bias b

Activation functions — the non-linearity

function	formula	range	notes
sigmoid	$\sigma(z) = 1/(1 + e^{-z})$	$(0, 1)$	probability-like; saturates → vanishing gradients
tanh	$\tanh(z)$	$(-1, 1)$	zero-centred; hidden-layer classic
ReLU	$\max(0, z)$	$[0, \infty)$	cheap, no saturation for $z > 0$ — today's default
softmax		$(0, 1)$, sums to	output layer for

function	formula	range	notes
	$e^{z_k} / \sum_j e^{z_j}$	1	multi-class

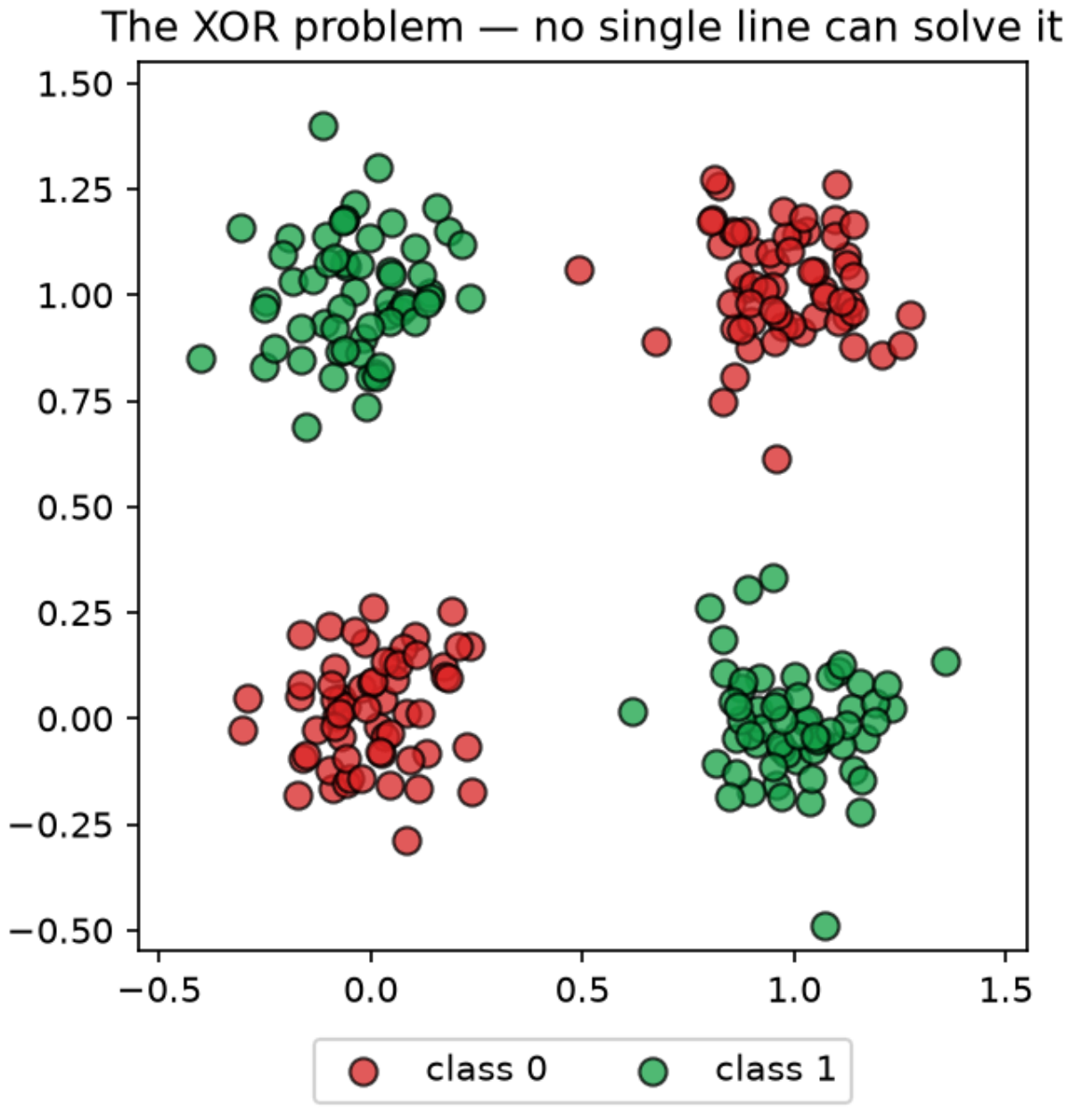
The non-linearity is the whole point: without it, stacking layers is still just a straight line. Compose enough of them and the network can approximate **any** smooth function.



The XOR problem — one neuron is not enough

A neuron draws a **single straight line** in input space. But some patterns simply cannot be split by a line — the classic example is **XOR**: the green class sits on the *diagonal*, the red class on the *anti-diagonal*.

No straight line separates them, so **a single perceptron can never solve XOR**. In 1969 Minsky & Papert proved this rigorously — and it triggered the first **AI winter**. The fix (a **hidden layer**) is the subject of the next session.


The XOR problem — no single line can solve it
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.5
0.0
0.5
1.0
1.5
class 0
class 1

Neural networks — a brief history

- **McCulloch–Pitts neuron** (1943) — the first mathematical model of a neuron
- **The perceptron** (Rosenblatt, 1957) — a learning algorithm!
- **The XOR problem** (Minsky & Papert, 1969) — the first AI winter
- **Backpropagation** (Rumelhart, Hinton & Williams, 1986) — a new hope
- Lacked data and compute; hard to train (1990s) — the second AI winter
- **AlexNet** (2012) — GPUs + ImageNet → the deep-learning revolution
- **Transformers / LLMs** (2017–) — the same ideas, at scale

Summary

- **Logistic regression** squeezes a linear score into a probability with the **sigmoid**; its boundary is a straight line (how it is *trained* — on the cross-entropy — comes in Session 2).
- Linear models and axis-aligned trees cannot capture **smooth, curved** boundaries — we need non-linear classifiers.
- A **neuron** is a weighted sum plus a non-linear **activation**; **logistic regression is one neuron**.
- The **activation function** supplies the non-linearity; without it a network collapses to a straight line.
- A single neuron cannot solve **XOR** — the fix is to **stack neurons in layers**, which we build next.

Next session: **multilayer networks** — how a hidden layer reshapes the data, and the **forward pass**.